

Pygame Meteor 예제 수업 메모

현재 진행 상황

학생들은 이미 다음 내용을 경험했다.

- Turtle 사용
- 반복문(loop)
- 함수(function)
- Pygame 기본 사용

Pygame에서는 다음과 같은 예제를 진행했다.

이미지 로드

```
meteor = pygame.image.load("meteor.png")
```

화면에 그리기

```
screen.blit(meteor, (meteor_x, meteor_y))
```

운석 이동

```
meteor_x += meteor_dx  
meteor_y += meteor_dy
```

벽에 튕기기

```
if meteor_x < 0 or meteor_x > 800:  
    meteor_dx *= -1  
  
if meteor_y < 0 or meteor_y > 600:  
    meteor_dy *= -1
```

운석 여러 개 만들기

처음에는

```
meteor_x  
meteor_y
```

```
meteor_dx
meteor_dy
```

만 사용했다.

그 다음

```
meteor_x2
meteor_y2
meteor_dx2
meteor_dy2
```

를 추가하여 운석을 2개로 늘렸다.

나중에는

```
meteor_x3
meteor_y3
meteor_dx3
meteor_dy3
```

처럼 계속 늘어나게 된다.

클래스 도입 아이디어

변수가 너무 많아지는 문제를 해결하기 위해 클래스를 소개한다.

첫 단계

```
class Meteor:
    pass
```

설명

- Meteor라는 새로운 종류를 만든다.
- 아직 기능은 없으므로 pass 사용

객체 생성

```
m1 = Meteor()  
m2 = Meteor()
```

각 객체에 데이터를 붙인다.

```
m1.x = 100  
m1.y = 100  
m1.dx = 3  
m1.dy = 2
```

```
m2.x = 300  
m2.y = 200  
m2.dx = -4  
m2.dy = 3
```

객체를 받는 함수

기존

```
meteor_x += meteor_dx  
meteor_y += meteor_dy
```

변경

```
def meteor_move(m):  
    m.x += m.dx  
    m.y += m.dy
```

사용

```
meteor_move(m1)  
meteor_move(m2)
```

설명

- 어떤 운석을 넣느냐에 따라 그 운석이 움직인다.

인스턴스 함수로 발전

기존

```
meteor_move(m1)
```

변경

```
class Meteor:
    def move(self):
        self.x += self.dx
        self.y += self.dy
```

사용

```
m1.move()
m2.move()
```

설명

- move 기능을 운석 안으로 넣는다.
- "m1아 움직여라"라고 읽을 수 있다.

이미지는 공유

질문

같은 이미지를 여러 번 로드하면?

```
img1 = pygame.image.load("meteor.png")
img2 = pygame.image.load("meteor.png")
```

결과

- Surface 객체 2개 생성
- 메모리 거의 2배 사용
- 로딩 시간도 2번 발생

추천

```
Meteor.image = pygame.image.load(  
    "meteor.png"  
)  
.convert_alpha()
```

사용

```
screen.blit(m1.image, (m1.x, m1.y))  
screen.blit(m2.image, (m2.x, m2.y))
```

설명

- 그림 파일은 하나
- 모든 운석이 공유

비유

운석은 100개지만 그림은 하나

이미지 공유 확인

```
print(id(m1.image))  
print(id(m2.image))
```

같은 숫자가 출력된다.

설명

- 두 객체가 같은 Surface를 사용 중

랜덤 운석 생성

```
import random  
  
m = Meteor(  
    random.randint(0, 800),  
    random.randint(0, 600),  
    random.choice([-5, -4, -3, 3, 4, 5]),  
    random.choice([-5, -4, -3, 3, 4, 5]),  
)
```

운석 100개 만들기

먼저 일반적인 방법

```
meteors = []

for i in range(100):
    m = Meteor(
        random.randint(0, 800),
        random.randint(0, 600),
        random.choice([-5, -4, -3, 3, 4, 5]),
        random.choice([-5, -4, -3, 3, 4, 5]),
    )

    meteors.append(m)
```

List Comprehension

위 코드를 한 줄로 표현

```
meteors = [
    Meteor(
        random.randint(0, 800),
        random.randint(0, 600),
        random.choice([-5, -4, -3, 3, 4, 5]),
        random.choice([-5, -4, -3, 3, 4, 5]),
    )
    for i in range(100)
]
```

설명

- 100번 만들고 리스트에 넣는 작업을 한 줄로 표현

앞으로 진행 예정

1. class Meteor: pass
2. m1.x, m1.y
3. meteor_move(m)
4. m.move()
5. Meteor.image 공유
6. 랜덤 생성

- 7. 리스트
 - 8. 윤석 100개 생성
 - 9. list comprehension
 - 10. 충돌 판정
 - 11. 점수 시스템
 - 12. 게임 오버
-

핵심 목표

학생들이

```
meteor_x
meteor_y
meteor_dx
meteor_dy

meteor_x2
meteor_y2
meteor_dx2
meteor_dy2
```

에서 벗어나

```
m1.x
m1.y

m2.x
m2.y
```

를 이해하고,

최종적으로

```
for m in meteors:
    m.move()
```

를 자연스럽게 사용할 수 있게 만드는 것이 목표이다.